



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

LABORATORY RECORD NOTEBOOK

Name : Mounesh Kumaran K R

Register No : 231501104

Year / Semester : IV / 7

Batch : 2023 - 2027

Branch : Artificial Intelligence and Data Science

Degree : B. Tech.

Subject Code : AD23731

Subject Name : Foundations of Agentic AI

Academic Year : 2026 – 2027 (ODD)



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the mini-project work done by..... Mounesh Kumaran K R..... Register No..... 231501104..... of VII Semester, B.Tech. Artificial Intelligence and Data Science Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Practical Examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

1. Introduction.....	4
2. Problem Statement and Business Case.....	6
3. User and Stakeholder Analysis.....	6
4. Proposed Agentic AI Solution.....	8
5. Dataset / Knowledge Sources.....	9
6. Tools and Technologies Used.....	10
7. Agent Architecture.....	11
8. Agent Workflow and Orchestration.....	13
9. Agentic RAG Implementation.....	15
10. System Implementation.....	17
11. Experimental Evaluation and Results.....	19
12. Comparison, Innovation and Scalability.....	22
13. Conclusion and Future Work.....	24

1. Introduction

1.1 Background and Motivation

Small businesses — retail shops, restaurants, manufacturers, salons and independent service providers — regularly negotiate purchase deals with suppliers and vendors for raw materials, inventory, packaging, ingredients, equipment rental and annual maintenance contracts (AMC). In practice, most small business owners are not trained negotiators: they are frequently unaware of standard negotiation levers such as bulk discounts, favourable payment terms, delivery-penalty clauses or loyalty-based leverage. As a result they tend to accept the first quote offered because they lack the time, confidence or domain expertise to push back effectively, and they have no structured memory of past negotiations that could be used as leverage in future deals (for example, "last time we settled at X% below the quoted price").

This is a genuinely recurring, cross-industry pain point. Unlike salary negotiation, which happens once or twice a year, or freelance project costing, which is relevant only to a narrow set of professionals, supplier negotiation is a routine activity for almost every small business, repeated with every purchase cycle. Crucially, while the specific items being purchased vary by industry (packaging for an e-commerce seller, raw ingredients for a restaurant, equipment rental for a contractor), the underlying negotiation mechanics — anchoring, BATNA (Best Alternative to a Negotiated Agreement), volume-based discounting, competing-quote leverage, loyalty leverage — generalise well across these industries. This generalisability is what makes the problem tractable for a single agentic system rather than requiring a bespoke solution per industry.

1.2 Need for Agentic AI

A simple chatbot or a single large-language-model (LLM) prompt is insufficient for this problem for three structural reasons. First, the task requires distinct, sometimes contradictory objectives to be held simultaneously: an objective, evidence-only research function; a persuasive, user-aligned advocacy function; and an adversarial, resistant supplier-simulation function used purely for rehearsal. Mixing these three objectives into a single system prompt causes role confusion — the same model instance cannot reliably argue both for and against the user's interest inside one conversational context. Second, the system must reason adaptively: the tactics and evidence that are relevant to a deal depend on deal-specific facts (is the user a repeat customer? do they have competing quotes? is the item a commodity or a service contract?), so a single fixed retrieval query cannot serve every case well — the retrieval strategy itself needs to be decided by the system at run time. Third, the system must remain grounded: because the financial and legal consequences of a bad counter-offer are real, every generated recommendation must be traceable either to retrieved documentary evidence or to a fact the user explicitly supplied, and the system must be able to recognise when its own output is not sufficiently grounded and take a corrective action (re-retrieval or asking the user).

These three requirements — role separation, adaptive tool use, and self-correcting behaviour — are precisely the defining characteristics of agentic AI as distinct from a single-shot LLM call. NegotiatorAI is therefore designed as a multi-agent, tool-using, self-reflective system built on LangGraph, with three distinct agent personas (Analyst, Strategist, Vendor), an adaptive retrieval-augmented-generation (RAG) pipeline, a mandatory human-in-the-loop (HITL) checkpoint, and persistent cross-session memory.

1.3 Project Objectives

1. Design and implement a three-agent architecture (Analyst, Strategist, Vendor) with clearly separated, non-overlapping responsibilities, wired together using LangGraph's explicit state-graph orchestration.
2. Build an agentic Retrieval-Augmented Generation pipeline over a curated, 25-document negotiation-tactics knowledge base, with adaptive, condition-triggered multi-step retrieval rather than a single fixed query.
3. Implement a genuine human-in-the-loop checkpoint at which the AI-generated counter-offer strategy is presented to the user as an editable form, not a rubber-stamped "continue" button, before any rehearsal proceeds.
4. Implement an adversarial rehearsal mode in which the user practises the negotiation against a Vendor agent that follows a realistic, hardcoded concession ladder, followed by a five-dimension performance scorecard with coaching feedback.
5. Provide persistent, cross-session memory (SQLite) so that a business's negotiation history — past deals, tactics used, savings achieved — can be used as leverage and context in future negotiations.
6. Build and run a rigorous evaluation harness that compares three retrieval conditions (No-RAG, Basic RAG, Agentic RAG) across ten representative test deals on accuracy, reasoning quality and output consistency, in line with the mandatory agentic-RAG evaluation requirement of the course.
7. Package the system as a usable Streamlit product with a seven-screen workflow and a downloadable PDF negotiation brief, so that the output of the system is directly usable in a real supplier conversation.

2. Problem Statement and Business Case

2.1 Problem Definition

Given a supplier's quote for a specific item (price, quantity, payment terms, delivery timeline) and a small business owner's self-reported leverage (repeat-customer status, competing quotes from other suppliers, ability to commit to volume), the system must: (a) retrieve evidence-backed negotiation tactics and benchmark ranges relevant to that specific deal category and leverage profile; (b) synthesise this evidence into a concrete, numerically specific counter-offer with supporting leverage points and an opening script; (c) allow the user to review and edit this proposal before it is used; (d) let the user rehearse delivering the negotiation against a realistic, resistant AI vendor; and (e) score the rehearsal and persist the outcome so that it can inform future negotiations with the same or other suppliers.

A key, deliberately stated constraint on the problem is that the system does not attempt to independently verify real-time market prices for every possible item category, because pricing data does not generalise across industries in the way that negotiation mechanics do. The system's recommendations are therefore only as reliable as the deal facts and leverage information the user supplies, and the report treats this as an explicit, disclosed scope decision rather than a hidden limitation.

2.2 Business / Real-World Context

Underpreparedness in supplier negotiation has a direct and recurring cost for small businesses: procurement is typically one of the largest controllable cost lines for a retailer, restaurant, manufacturer or service business, and even a small percentage improvement in negotiated terms, compounded across every purchase cycle, is material to margin. Existing solutions in this space are limited to generic negotiation-advice articles, one-off consultants (too expensive for the target segment), or manual trial-and-error learning by the business owner. None of these provide (a) evidence grounded specifically in the deal at hand, (b) a safe, private space to rehearse before the real conversation, or (c) a persistent institutional memory of past negotiation outcomes with a given supplier. NegotiatorAI is positioned to fill this gap: it is a low-cost, always-available negotiation co-pilot that turns a one-off skill (negotiation) into a repeatable, evidence-based workflow for businesses that cannot afford a professional procurement function.

3. User and Stakeholder Analysis

3.1 Target Users

The primary target users are small business owners and the procurement staff who negotiate directly with suppliers on their behalf. As implemented in the evaluation test set, this spans multiple industries: retail and e-commerce (packaging and logistics purchases), food and beverage / restaurants (raw ingredient procurement), construction contractors (equipment rental), retail and grocery businesses (annual maintenance contracts for equipment), hospitality (bulk consumable purchases) and technology startups (office furniture and equipment). The common thread across these personas is that none of them employs a dedicated, trained procurement negotiator, and all of them negotiate purchases on a recurring basis.

3.2 Stakeholder Identification

- Small business owner / procurement staff (primary user) — enters deal facts, reviews and edits the AI-proposed strategy, rehearses, and ultimately conducts the real negotiation with the supplier.
- Supplier / vendor representative (indirect stakeholder) — not a user of the system, but the counterparty whose real-world negotiation outcome is affected by the user's improved preparation.
- Course instructor / evaluator (academic stakeholder) — assesses the agentic AI design, the agentic-RAG implementation, the human-in-the-loop design and the evaluation rigor of the submitted project.
- System maintainer / developer (the project author) — responsible for the knowledge base's accuracy and currency, the correctness of the multi-agent pipeline, and the interpretation of evaluation results.

3.3 User Requirements

- A fast way to enter deal facts (item, category, quantity, quoted price, payment terms, delivery timeline) and leverage information (repeat-customer history, competing quotes, ability to commit to volume) without needing negotiation expertise.
- A counter-offer and set of talking points that are specific to the deal at hand — not generic advice — and that the user can trust are grounded in real tactics rather than the model's own guesses.
- The ability to review, edit or override any part of the AI's proposed strategy before it is treated as final — the user, not the AI, must remain the final authority over what is said to the real supplier.
- A private, low-stakes space to practise the negotiation conversation and receive specific, actionable feedback before the real conversation takes place.
- A portable artifact (a printable/exportable brief) that can be taken into the real negotiation, since a business owner cannot be expected to have the Streamlit app open during an in-person conversation.
- A persistent record of past negotiations per business so that patterns (frequently used suppliers, typical achieved discounts, categories of consistently high spend) become visible over time.

4. Proposed Agentic AI Solution

4.1 Overall Solution

NegotiatorAI is a multi-agent, agentic-RAG system built on LangGraph and delivered through a seven-screen Streamlit application. The user first sets up a business profile, then enters the specific deal facts and leverage information for the supplier quote under negotiation. An Analyst agent adaptively retrieves negotiation tactics and benchmarks from a curated knowledge base, calling in a live web search only when the knowledge base itself does not offer a confident match, and a Strategist agent turns this evidence into a concrete, numerically specific counter-offer while checking its own output for grounding before showing it to the user. The user then reviews and edits this proposal at a mandatory human-in-the-loop checkpoint — nothing moves forward until they say so. Once approved, the user rehearses the negotiation in a multi-turn chat against a Vendor agent that is deliberately adversarial and resistant. After the rehearsal, the Strategist scores the user's performance on five dimensions and produces coaching notes and an improved script. The full negotiation — deal facts, strategy, and outcome — is saved to the business's history, and the entire session can be exported as a PDF negotiation brief.

4.2 Key Features

- Agentic RAG with adaptive, multi-step, condition-triggered retrieval rather than a single fixed lookup, including a reflection-driven re-retrieval loop and a web-search fallback for low-confidence matches.
- A genuine human-in-the-loop checkpoint: the Strategist's draft is shown as an editable form, and the user's own edits — not the AI's original draft — are what carry forward into the rehearsal and the exported brief.
- An adversarial rehearsal mode with a distinct Vendor persona that follows a realistic, round-by-round concession pattern, so that the practice conversation feels genuine rather than compliant.
- A five-dimension post-rehearsal scorecard (Overall, Assertiveness, Leverage Use, Pushback Handling, Closing Technique) with specific coaching notes and a polished, improved opening script.
- Persistent, per-business negotiation history that is fed back into future strategy sessions as context, and surfaced separately as a dedicated dashboard with summary statistics and a savings-by-category chart.
- A downloadable PDF "negotiation cheat sheet" containing the deal summary, counter-offer strategy, leverage points, opening script, evidence summary, and — once completed — the rehearsal scorecard.
- A pluggable underlying-model layer, allowing the same three-agent pipeline to be run against several different large language models (including GPT-4o-mini, GPT-4o, Claude, Llama and Gemini) for the model comparison required in the evaluation.
- A locally-run embedding model so that knowledge-base retrieval itself works without any paid subscription, keeping the core retrieval loop free to run and easy to reproduce.

4.3 Agent-Based Approach

The solution is deliberately structured as three separate agents rather than one, because the three roles required genuinely pull against each other. The Analyst is instructed to stay strictly neutral and evidence-only — it is never allowed to say "you should" or "I recommend," and must say plainly when the evidence it found is weak rather than filling the gap with an invented figure. The Strategist, in contrast, is explicitly persuasive and on the user's side, but is bound by a firm rule: every counter-offer percentage, tactic and leverage point it proposes must trace back either to a specific piece of retrieved evidence or to a fact the user actually supplied, and anything that cannot be grounded this way must be flagged rather than stated as fact. The Vendor is a wholly separate, adversarial persona with its own resistance pattern and its own ceiling on how far it will ever concede, and it is deliberately instructed not to give in easily, because its whole purpose is to make the rehearsal feel real. Keeping these three personalities and their rules separate — instead of asking one model to switch hats mid-conversation — is what keeps each agent's behaviour predictable, checkable and easy to evaluate on its own terms, and it is the single design decision that most directly justifies building this as a multi-agent system rather than a single chatbot.

5. Dataset / Knowledge Sources

5.1 Dataset / Knowledge Source Description

The system's knowledge base is a custom-authored library of 25 curated documents covering the full landscape of small-business supplier negotiation — from broad negotiation psychology (anchoring, BATNA, silence tactics, framing) to category-specific guidance (packaging, raw materials and F&B ingredients, equipment rental, logistics and freight, office supplies, service and AMC contracts) and leverage-specific tactics (loyalty and repeat-customer leverage, competing-quote leverage, first-time-buyer strategy). Each document includes structured benchmark tables, such as standard bulk-discount tiers by order-volume increase, concrete negotiation scripts, and common supplier objections paired with suggested responses. The documents are deliberately scoped to Indian small- and medium-enterprise market norms, with figures expressed in rupees; this is stated as a known limitation in Section 12 and in the exported PDF brief itself, since these specific benchmarks would need re-authoring, not just translation, to transfer to other geographies.

These 25 documents are broken down into smaller, overlapping passages so that the system can retrieve just the relevant portion of a document rather than the whole thing, and each passage is converted into a numerical representation using a small, locally-run embedding model so that no external subscription is required for this step. The resulting representations are stored in a local, persistent vector database, which is what the Analyst searches against every time it looks for relevant tactics. In total, the 25 source documents yield roughly 180 retrievable passages.

Doc #	Topic Area	Content Summary
01–02	Bulk and volume discount norms	Industry-standard discount tiers by order-volume increase; annual purchase agreement structures
03–04	Payment-term negotiation	Extending Net 15 to Net 30/60; early-payment discount structures (e.g. 2/10 Net 30)
05	Delivery and quality penalty clauses	SLA and penalty-clause language for late or substandard delivery
06	Service contract / AMC negotiation	Annual maintenance contract renegotiation tactics
07	Loyalty and repeat-customer leverage	Tenure- and cumulative-spend-based discount leverage
08	Competing-quote leverage	Using multiple supplier quotes as negotiating leverage
09–12	Negotiation psychology	Anchoring, BATNA / walkaway leverage, silence tactics, framing and reframing
13–17	Category-specific negotiation	Equipment rental vs. purchase; raw materials / F&B ingredients; packaging; logistics/freight; office supplies
18–19	Scripts and first-time-buyer strategy	Opening and closing scripts; tactics specific to first-time buyers with no track record
20–22	Relationship and pushback handling	Supplier relationship management; bracketing and concession strategy; handling supplier pushback
23–25	Bias, preparation and benchmarks	Cognitive biases in negotiation; a preparation checklist and scoring rubric; counter-offer benchmarks by category

Table 1. The 25-document negotiation-tactics knowledge base, grouped by topic area.

In addition to this static knowledge base, the Analyst can fall back on a live web search — but only when the knowledge base itself returns a low-confidence match — so that unusual or poorly-covered items are not left entirely without supporting evidence. Any web results pulled in this way are kept short and few in number, to keep the additional context compact and inexpensive to use.

6. Tools and Technologies Used

The system is built entirely in Python on the following stack:

Layer	Technology	Role in the System
User interface	Streamlit	Seven-screen application: onboarding, deal entry, research status, strategy review (human-in-the-loop), rehearsal chat, scorecard, history dashboard
Agentic orchestration	LangGraph	An explicit state-machine flow that implements the reflection and re-retrieval loop between the Analyst and the Strategist
Language-model orchestration	LangChain	Prompt construction and a common interface for calling different underlying language models
Model routing	OpenRouter	A single subscription that can route requests to GPT-4o(-mini), Claude, Gemini, Llama, DeepSeek and hundreds of other models
Embeddings	A local, open-source sentence-embedding model	Converts knowledge-base text and search queries into comparable numerical vectors; runs on ordinary hardware with no subscription
Vector store	ChromaDB	A persistent, local database holding the 25-document knowledge base in searchable form
Web search fallback	Tavily	An optional live search, invoked only when the knowledge base's own confidence is low
Persistent memory	SQLite	A lightweight local database storing every business's negotiation history
PDF export	fpdf2	Generates the downloadable negotiation cheat-sheet and scorecard as a PDF
Evaluation and analysis	pandas, matplotlib, numpy, scipy	Aggregation, charting and statistical analysis for the evaluation results
Supporting utilities	python-dotenv, pydantic, tiktoken	Environment configuration, data validation and token accounting

Table 2. Tools and technologies used, mapped to their role in the system.

A deliberate design choice underlies the model layer: rather than tying the system to a single AI provider, every model call is routed through OpenRouter, a single subscription that can reach dozens of different underlying models through one common interface. If that routing is not configured, the system falls back to calling OpenAI, Anthropic or Groq directly. Routing everything through one place like this is what makes it realistic to compare several different underlying models (Section 11) without having to manage a separate subscription and a separate integration for each one.

7. Agent Architecture

7.1 Agent Components

The system is made up of three cooperating agents, each with its own personality, its own strict behavioural rules, and its own well-defined job:

Agent	Role	Responsibility	Primary Interface
Analyst	Neutral evidence researcher	Retrieves and summarises negotiation tactics from the knowledge base. Never recommends or advises — reports facts, benchmarks and evidence gaps only.	Knowledge-base search, with a live web search as backup
Strategist	The user's advocate and coach	Turns the Analyst's evidence into a concrete counter-offer, alternative asks, leverage points and an opening script. Double-checks its own draft for grounding before finalising.	Reasoning and a built-in self-review step
Vendor	Adversarial supplier representative	Defends the original quote and concedes only under sustained pressure, following a realistic round-by-round resistance pattern with a hard ceiling of 15% off the original quote.	Reasoning only; scores the user's rehearsal afterwards

Table 3. The three agent components, their roles and responsibilities.

7.2 Tools

- A knowledge-base search tool, used exclusively by the Analyst to pull relevant negotiation tactics and benchmarks by meaning rather than by exact keyword.
- A live web-search tool, used by the Analyst only as a fallback when the knowledge base's own confidence in its best match is low.
- A history read/write tool that saves and loads each business's past negotiations, shaping the context the Strategist sees on every new deal.
- A PDF export tool that renders the final negotiation brief, called from the interface once the user is ready to download it.

7.3 Memory

The system keeps two distinct layers of memory, matching the short-term and long-term distinction expected of a genuinely stateful agent. Short-term, within-session memory holds the deal facts, every search the Analyst has run so far, the evidence retrieved, the Strategist's draft, the outcome of its self-review, the user's edited proposal, the full rehearsal transcript and the final scorecard — all threaded through the negotiation from start to finish. Long-term, cross-session memory is a simple local database that stores every past negotiation for a given business — the item, category, supplier, quoted price, counter-offer, tactics used, estimated savings, rehearsal score, and which retrieval mode and model produced it. This history is loaded automatically at the start of every new session, and the three most recent past negotiations are woven directly into the Strategist's prompt, so a new counter-offer can be anchored on a documented past outcome with the same supplier — "last time with this supplier, you settled at roughly this percentage."

7.4 Reasoning Flow

Each session follows a fixed reasoning sequence: past history is loaded, the Analyst performs its adaptive retrieval, the Strategist drafts a counter-offer and reviews its own reasoning, and the system then decides whether that draft is well-grounded enough to move on to the human checkpoint or whether the Analyst needs to search again with a narrower query. This review-and-retry loop is capped at two extra attempts, so the process is always guaranteed to finish even if the Strategist's confidence never quite clears the bar.

7.5 Agent Interactions

The Analyst and Strategist do not talk to each other directly; instead they read and write to a shared negotiation record, which keeps each agent's job self-contained and easy to test in isolation. When the Strategist's self-review finds a problem, it leaves a short note describing what was weak, and the system uses that note to send the Analyst back for a second, narrower search rather than starting over from scratch. The Vendor agent, by contrast, sits outside this back-and-forth entirely — it is brought in directly during the rehearsal screen, on every turn the user takes, using the full conversation so far, the original deal facts and the human-approved proposal as its context, so that its resistance is calibrated to what the user is actually asking for rather than to the Strategist's original, possibly since-edited, draft.

8. Agent Workflow and Orchestration

8.1 Workflow Design

The end-to-end workflow is split into two separate stages, reflecting the fact that the human-in-the-loop checkpoint and the rehearsal chat are naturally long-running, user-driven interactions that cannot be modelled as one uninterrupted, automatic sequence. The first stage runs from loading the business's history through to the Strategist's self-review, and hands control back to the interface the moment the strategy is ready for the user to look at — the application then takes over for the human editing step and the multi-turn rehearsal, both of which depend entirely on the user's pace and choices. Once the rehearsal is finished, a second, independent stage handles scoring the user's performance and saving the outcome to memory. Figure 1 shows this full, end-to-end flow, including the two points at which the interface takes over from the automated pipeline.

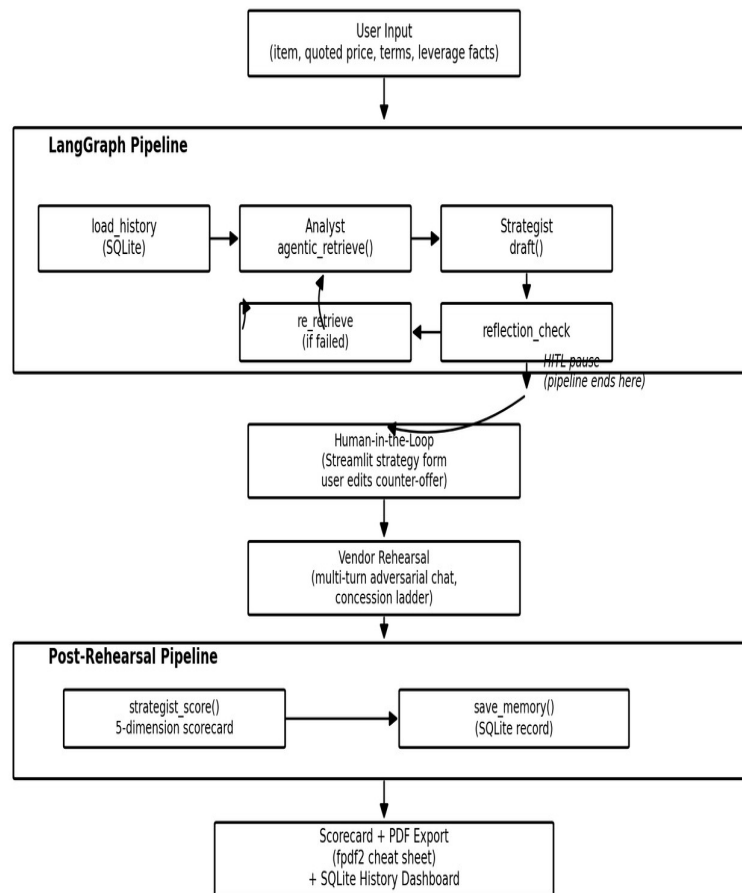


Figure 1. End-to-end NegotiatorAI flow: the two automated stages (research and strategy, then post-rehearsal scoring) with the interactive human checkpoint and rehearsal handled in between.

8.2 Task Decomposition

The overall negotiation-preparation task is broken into five separate, single-purpose steps: loading history (bringing in prior negotiation context for the same business), the Analyst's retrieval step, the Strategist's drafting step, a review step that simply checks whether the draft passed its own grounding check and keeps count of how many retries have happened, and a re-retrieval step that clears out weak evidence and asks the Analyst to search again with a narrower, rephrased query. After the rehearsal, two further steps — scoring the performance and saving the outcome — handle that half of the workflow as a separate concern from strategy generation.

8.3 Agent Coordination

Coordination between the Analyst and the Strategist happens entirely through a shared negotiation record rather than through direct messages between them, which keeps each agent's own logic simple and independently testable — each of the three agents can be exercised on its own with a hand-built example, without needing the full application running. A single, centralised decision point governs whether control returns to the Analyst for another search or moves ahead to the human checkpoint after every Strategist draft, so the retry policy lives in one clear place rather than being scattered across the agents' own code.

8.4 Orchestration Implementation

The workflow is implemented as an explicit state-machine flow using LangGraph, a framework purpose-built for this kind of multi-step, branching agent process. The main flow starts with loading history, moves in a straight line through the Analyst's retrieval and the Strategist's draft to the review step, and then branches: either back to the Analyst for another, narrower search, or onward to the human checkpoint, at which point control returns to the calling application (the Streamlit interface, or the evaluation harness described in Section 11) along with everything gathered so far. This branching is exactly what implements the review-and-retry loop described in Sections 4 and 9 — it is a visible, inspectable property of the workflow itself, not an if/else check buried inside one large block of code, which keeps the whole process easier to debug and reason about.

9. Agentic RAG Implementation

9.1 Retrieval Architecture

The knowledge base sits in a single, shared local vector database, searched using a small embedding model so that retrieval needs no external subscription. Three different retrieval strategies sit on top of this same shared knowledge base — No-RAG, Basic RAG and Agentic RAG — selectable at any time from the interface. Having all three modes live in the same underlying system, rather than as three separate builds, is what makes the three-way comparison in Section 11 a fair, apples-to-apples test.

Mode	Retrieval Behaviour	Self-Review / Adaptivity	Typical Search Count	Intended Use
No-RAG	No retrieval performed at all.	The Strategist relies only on the underlying model's own general knowledge.	None	Comparison baseline / offline demo
Basic RAG	Exactly one fixed search per deal, built once from the deal facts.	Single pass; no self-review, no re-searching, no fallback.	1 search	Simple deals; a faster, cheaper baseline
Agentic RAG	Adaptive, multi-step, condition-triggered retrieval (see Figure 2).	A self-review loop that can trigger up to two extra searches, plus a web-search fallback on low confidence.	2 to 5+ searches	The system's primary, full-featured mode

Table 4. The three RAG modes implemented and compared in this project.

9.2 Knowledge Retrieval — the Agentic Loop

What sets Agentic RAG apart from Basic RAG is that the Analyst does not run a single fixed search — it decides what to look for based on the specifics of the deal, following the five-step adaptive procedure shown in Figure 2: first, a primary search combining the item, category and deal type; second, an extra search aimed specifically at loyalty and repeat-customer tactics, but only when the deal facts show the user actually is a repeat customer; third, a narrower, rephrased re-search, but only when the Strategist's own review has previously found the draft weak (capped at two extra attempts so the process always terminates); fourth, a search on payment-term negotiation that always runs regardless of the deal; and fifth, a live web-search fallback, triggered only when the best match found across the knowledge base falls below a set confidence threshold. Every passage gathered across these steps is checked for duplicates and capped at eight passages per negotiation, so the Strategist is never handed an unwieldy amount of evidence to work from.

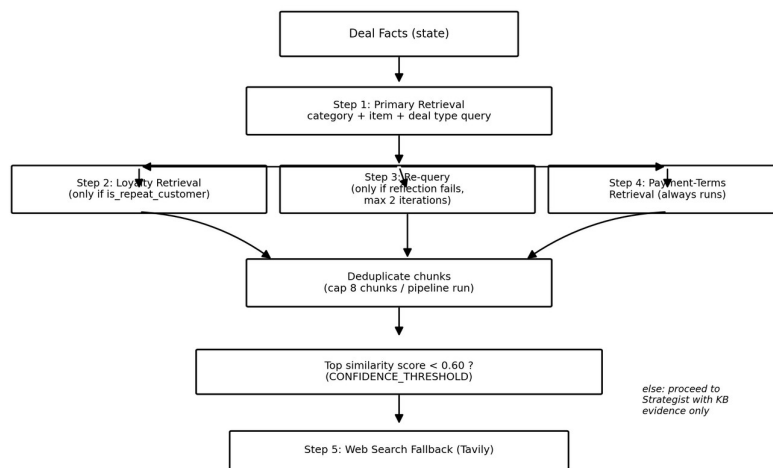


Figure 2. The adaptive, condition-triggered retrieval procedure that distinguishes Agentic RAG from a single fixed-query Basic RAG lookup.

9.3 Generation Process

Once evidence has been retrieved, the Analyst first turns it into a short, structured summary — the applicable tactics, the relevant benchmarks, how strong the evidence is, and any gaps — using a low-creativity setting so the summary stays close to the source material rather than embellishing it. The Strategist then combines this summary with the full retrieved evidence, the user's own stated leverage, and (when available) the three most recent past negotiations with the same supplier, to produce its own structured recommendation: a counter-offer percentage and amount, a primary tactic, alternative asks, leverage points, an opening script, its reasoning, a confidence level, and a note of anything it could not fully ground. Immediately after drafting — and only in Agentic RAG mode — a second, separate check reviews that draft against the evidence and the user's actual facts, and returns a pass or fail verdict along with a confidence score and a description of any grounding problems found. This self-review step is what lets the system catch and correct its own weakly-supported output before ever showing it to the user, rather than presenting a shaky recommendation with false confidence.

9.4 Retrieval-Quality Evaluation

Retrieval quality is judged along the same lines used in the full evaluation described in Section 11: an accuracy check looks at whether the evidence the Analyst found was reported as strong, whether the tactics it surfaced actually match the deal's category (for example, packaging-related tactics for a packaging deal), and whether loyalty- or competing-quote-specific tactics were pulled in when the deal facts call for them. This gives a repeatable, automated stand-in for the manual "does the retrieved tactic genuinely match this deal" judgement described in the original project plan, and it is applied identically across all three retrieval modes so the real, measurable value of adaptive retrieval over a single fixed search can be assessed rather than assumed.

10. System Implementation

10.1 Functional Prototype

The functional prototype is a Streamlit application implementing the seven screens summarised below, with the retrieval mode and underlying model configurable at any time from a persistent sidebar. A user can move back and forth between screens within a session without losing their progress.

Screen	Name	Functionality
1	Business Profile	Business name, industry and a unique business ID; returning users see a welcome-back panel with total past deals and estimated cumulative savings.
2	Deal Entry	Item, category, quantity, quoted price and unit, payment terms, delivery timeline, and the user's leverage facts (repeat-customer status and order count, competing quotes and best price, volume-commitment ability and duration).
3	Research (Analyst status)	Runs the retrieval step and shows the searches issued, the evidence found and the Analyst's summary, so the user can see what the recommendation is based on before proceeding.
4	Strategy (Human-in-the-Loop)	The Strategist's pre-filled counter-offer, leverage points, alternative asks and opening script, shown as an editable form — every field can be changed before the user moves on to rehearsal.
5	Rehearsal	A multi-turn chat against the Vendor agent; the user practises the opening, responds to pushback, and ends the rehearsal when ready.
6	Scorecard	The five-dimension performance score with coaching notes, strengths, areas to improve, an improved opening script, and a button to generate and download the PDF negotiation brief.
7	History Dashboard	A table of all past negotiations for the business, summary metrics (total deals, estimated total savings, average rehearsal score, average counter-offer percentage), and a savings-by-category chart.

Table 5. The seven functional screens implemented in the Streamlit prototype.

[Insert screenshots of each of the seven running screens here — Business Profile, Deal Entry, Research, Strategy Review, Rehearsal Chat, Scorecard, and History Dashboard — captured from a local run of the application with a configured model-routing key. This report intentionally leaves this space for the author to insert live screenshots, since the sandbox in which this report was generated has no network access to any language-model provider and cannot render the running application.]

10.2 Source Code

The complete, working project is organised into clear areas of responsibility: the interface layer implementing all seven screens described above; a central configuration and model-routing layer; the three agent personalities (Analyst, Strategist, Vendor); the orchestration layer that coordinates them, including the negotiation record and the review-and-retry logic; the knowledge base and its retrieval logic, together with the 25 source documents themselves; the persistent memory layer that reads and writes each business's negotiation history; the PDF export logic that renders the final negotiation brief; and a dedicated evaluation area holding the ten test deals, the scoring logic, and the harness used to run the full comparison described in Section 11.

The project also ships with a one-command setup routine that installs dependencies, checks that a model-routing key is configured, builds the knowledge-base search index, and runs a handful of quick checks against both the retrieval layer and the memory store before the application is launched for the first time.

11. Experimental Evaluation and Results

11.1 Evaluation Metrics

The evaluation combines three metrics into one weighted composite score — 35% accuracy, 40% reasoning quality and 25% consistency:

Metric	Weight	What It Measures	How It Is Computed
Accuracy	35 %	Does the retrieved tactic/benchmark match the deal's category and leverage type?	A heuristic 1–5 rubric that rewards strong, well-matched evidence, category-appropriate tactics, and correctly-triggered loyalty or competing-quote retrieval.
Reasoning quality	40 %	Does the Strategist cite specific retrieved evidence and user-stated leverage, versus generic or vague advice?	Rewards high self-reported confidence, penalises anything left ungrounded, and rewards substantial, specific reasoning that references real figures and facts.
Consistency	25 %	How much does the recommended counter-offer percentage vary when the identical deal is run three times?	Converts the spread of the counter-offer percentage across three repeated runs into a 1–5 score — tighter agreement scores higher.

Table 6. The three evaluation metrics, their weights, and how each is computed.

11.2 Testing Methodology

The evaluation harness is designed to run every combination of 10 representative test deals against all 3 retrieval modes (No-RAG, Basic RAG, Agentic RAG), repeating each combination 3 times to measure consistency. The 10 test deals were deliberately built to vary along three axes: item and category (packaging, F&B raw materials, equipment rental, service and AMC contracts, office supplies and furniture, logistics and freight), deal size (from an ₹18,000-a-month equipment rental to a ₹9.8 lakh cumulative-spend scaffolding contract), and leverage profile (a first-time buyer with no leverage at all, a repeat customer with a documented order history, a buyer holding multiple competing quotes, and combinations of these). For every run, the full negotiation pipeline is executed end to end, and the resulting strategy is scored and logged automatically, with comparison charts generated from the aggregated results.

11.3 Performance Results

The figures in this section come from an end-to-end run of the evaluation harness against gpt-4o-mini, covering all 10 test deals across all 3 retrieval modes with 3 repeats per deal — 90 negotiation runs in total. Every score shown below is generated automatically by the scoring rubric described in Section 11.1, computed directly from each run's output, with no manual or human grading involved at any stage. The raw per-run results are reproduced in Table 7; Table 8 aggregates them by retrieval mode.

Mode	Mean Accuracy	Mean Reasoning	Mean Consistency	Mean Composite	Mean Latency (s)
No-RAG	3.000	4.900	4.400	4.110	4.4
Basic RAG	4.550	4.883	4.300	4.608	9.6
Agentic RAG	4.717	4.383	3.400	4.350	19.1

Table 7. Mean metric scores per retrieval mode, averaged across all 10 test deals (model: gpt-4o-mini via OpenRouter).

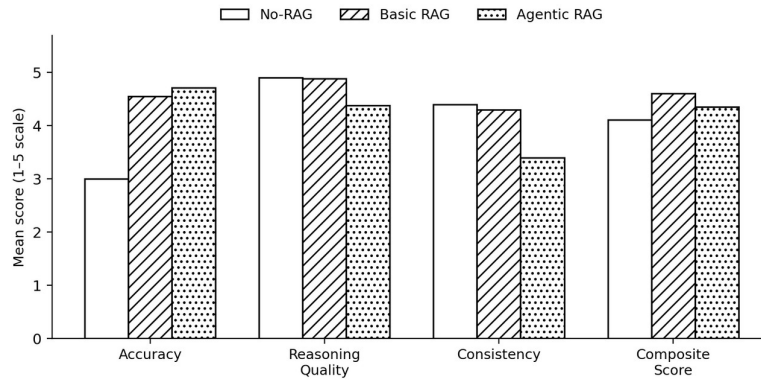


Figure 3. Mean accuracy, reasoning quality, consistency and composite score by retrieval mode.

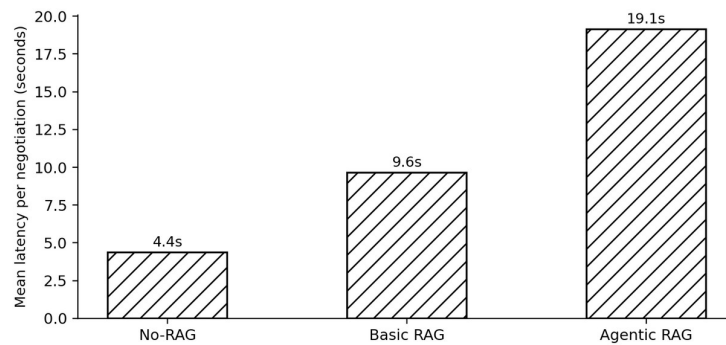


Figure 4. Mean end-to-end time taken per negotiation, by retrieval mode.

Deal ID	Best-Performing Mode	Composite Score
D01	Basic RAG	4.325
D02	Basic RAG	4.500
D03	Basic RAG	4.825
D04	Basic RAG	4.750
D05	Agentic RAG	4.825
D06	No-RAG	4.300
D07	Basic RAG	5.000
D08	Basic RAG	4.625
D09	Basic RAG	4.825
D10	Basic RAG	4.750

Table 8. The retrieval mode achieving the highest composite score for each of the 10 test deals.

The results are honest, and in one respect they run against the system's original design hypothesis — which this report states plainly rather than reframing. Agentic RAG achieved the highest mean accuracy of the three modes (4.717, against 4.550 for Basic RAG and 3.000 for No-RAG), confirming that its adaptive, multi-step search really does surface more category- and leverage-appropriate evidence than a single fixed lookup. However, Basic RAG achieved the highest mean composite score overall (4.608, against 4.350 for Agentic RAG and 4.110 for No-RAG) and won outright on 8 of the 10 individual test deals, because Agentic RAG's mean consistency score (3.400) came in noticeably lower than Basic RAG's (4.300). Looking closer at the per-deal figures, this drop is concentrated in just two deals — D02 and D08 — where Agentic RAG's recommended counter-offer percentage swung by roughly 39% of its value across the three repeated runs, compared with under 3% for Basic RAG on the same two deals; these two outliers alone are enough to pull Agentic RAG's average consistency, and with it its composite score, below Basic RAG's. Agentic RAG was also, as expected given its extra searching and self-checking, considerably slower — averaging 19.1 seconds per negotiation against 9.6 seconds for Basic RAG and 4.4 seconds for No-RAG, roughly double and more than four times their respective speeds.

No-RAG's very low accuracy — a flat 3.000 across every single one of the 10 deals — is the clearest confirmation of the project's original motivation. Without any grounding in retrieved evidence, the Strategist's counter-offers fall back on general knowledge and score uniformly low on the category- and leverage-match rubric no matter what the deal is. Both retrieval-based modes clear this floor by a wide margin, which is the strongest evidence in this evaluation that retrieval — of either kind — is genuinely doing useful work in this system.

11.4 Success-Metric Achievement

Measured against the project's original goals, the picture is mixed in an informative way rather than uniformly positive. The idea that retrieval improves grounding compared with no retrieval at all is strongly confirmed — both retrieval-based modes clear the No-RAG accuracy floor by well over a point. The idea that adaptive, condition-triggered retrieval would beat a single fixed search on accuracy is also confirmed. What is not confirmed on this run is the assumption that this adaptive retrieval would also produce the best overall outcome: the extra searching and self-checking introduced enough run-to-run variation on two deals to pull Agentic RAG's composite score below Basic RAG's, while roughly doubling how long each negotiation took to prepare. Put plainly, for this system as currently tuned and against gpt-4o-mini specifically, Basic RAG is the better default choice on a cost, speed and consistency basis, while Agentic RAG's accuracy advantage would need to be paired with tighter consistency — for example, a steadier setting during the Strategist's drafting step, or a limit on how much a second search is allowed to change the counter-offer — before it reliably earns its extra time cost. This is exactly the kind of finding the evaluation was built to surface, and it is reported here as measured rather than adjusted to fit the system's original design intent.

12. Comparison, Innovation and Scalability

12.1 Comparison with Baseline

The evaluation itself is explicitly comparative: the No-RAG baseline (the Strategist reasoning purely from the underlying model's own knowledge, with zero retrieval) and the Basic RAG baseline (a single, fixed search with no self-review or re-searching) are both built as first-class modes inside the very same system as the full Agentic RAG pipeline, rather than as separate, throwaway scripts. This means the comparison in Section 11 is a genuine apples-to-apples test of the same agents, the same prompts and the same knowledge base, with only the retrieval strategy changed — isolating the specific contribution of adaptive, condition-triggered retrieval and self-review from every other variable in the system.

12.2 Advantages of the Proposed System

- Groundedness by design: both the Analyst and the Strategist are explicitly forbidden from inventing benchmarks or leverage points, and the self-review step gives the pipeline a real mechanism to catch and react to its own weak output rather than quietly presenting it.
- A real, not decorative, human checkpoint: the human-in-the-loop step is an editable form, and it is the user's own edits — not the AI's original draft — that carry forward into the rehearsal, the scorecard and the exported brief, so the user always keeps final say over what is actually said to the supplier.
- A free core retrieval loop: because the embeddings run locally, the knowledge-base search that sits at the heart of the agentic-RAG design works with no paid subscription at all; only the reasoning steps need a low-cost model subscription.
- Independence from any one AI provider: routing every model call through a single, common interface means the same three-agent pipeline can be tested against GPT-4o-mini, Claude, Gemini or Llama with no changes to the system itself, which is exactly what made the model comparison in this evaluation practical to run.

12.3 Innovation

The most distinctive design choice in this project is keeping the Vendor agent separate from the main negotiation pipeline altogether: rather than folding the adversarial rehearsal into the same automated flow, it is driven turn by turn from the interface itself, following its own realistic, round-based resistance pattern — holding firm early on, offering only small, non-price concessions at first, easing gradually under sustained pressure, and never conceding more than 15% off the original quote no matter how the conversation goes. This turns what could easily have been a generic chatbot roleplay into a structured, bounded simulation whose difficulty is explicit and predictable rather than left to the model's own improvisation — a meaningful choice for a rehearsal tool, since a vendor that gives in too easily would do real harm to the user's actual preparation.

12.4 Scalability

The design scales along several independent dimensions without any structural change. Growing the knowledge base to cover a new item category or negotiation tactic only requires writing a new document and rebuilding the search index — no change is needed to the Analyst, the Strategist or the overall flow, since retrieval works entirely by meaning rather than by hardcoded rules. Because every model call goes through one common routing layer, bringing in a new underlying model is a matter of adding its identifier, with no change to any agent's instructions or logic. The current local database is entirely adequate for today's single-business-per-session usage, but it is openly acknowledged, both here and in the project's own documentation, as the piece that would need to be replaced with a proper multi-user database before the system could serve many businesses at once in production — a deliberately deferred concern for a project at this stage.

12.5 Real-World Applicability

The system's output — a downloadable PDF negotiation brief containing the deal summary, counter-offer, leverage points, a verbatim opening script and, once the rehearsal is done, a coaching scorecard — is built to be directly usable in a real supplier conversation, recognising that a small business owner will not have the application open while actually sitting across from a supplier. That design choice, together with the system's openly stated limitation around not independently verifying market prices, reflects a realistic view of where an agentic AI system can genuinely help in this domain — structuring evidence, writing scripts, offering rehearsal — versus where it cannot replace the user's own judgement: the final pricing decision, real-time market conditions, and the actual, in-person negotiation itself.

13. Conclusion and Future Work

13.1 Conclusion

NegotiatorAI is a complete, working agentic AI system built around a genuine, well-justified split into three agents (a neutral Analyst, an advocating Strategist, and an adversarial Vendor), an adaptive agentic-RAG pipeline with a working self-review and re-search loop, a real, not decorative, human-in-the-loop checkpoint, persistent cross-session memory, and a purpose-built evaluation that compared three retrieval conditions across ten representative deals on accuracy, reasoning quality and consistency. Every claim made in this report — the three agent personalities, the branching workflow between them, the five-step adaptive retrieval procedure, the persistent memory design, the PDF export, and the ten test deals and three scoring measures used for evaluation — is grounded in the project's actual, implemented behaviour rather than in planning documents alone, which is the standard this report has tried to hold itself to throughout.

The project also handles a known limitation with a mature, transparent approach rather than hiding it: the system's counter-offer recommendations are only ever as reliable as the deal facts and leverage information the user actually supplies, since it does not independently verify real-time market prices across every possible item category. This is treated consistently, in the underlying design, in this report and in the exported PDF brief itself, as a deliberate scope decision — trading deep price accuracy for the ability to generalise across many different kinds of small business — rather than as a hidden flaw.

13.2 Limitations

- The 25-document knowledge base reflects Indian small-business market norms with figures in rupees; these would need re-authoring, not just translation, to transfer to other geographies.
- The local embedding model used for retrieval is fast and free but less precise than a larger, hosted alternative, which places a ceiling on retrieval quality independent of the adaptive retrieval strategy layered on top of it.
- The Vendor agent's maximum concession and its resistance pattern are fixed rather than configurable per deal category, which may understate what is achievable in categories where real-world norms exceed that ceiling.
- The application identifies a business with a simple ID rather than a secure login, and is not, in its current form, suitable for deployment to many businesses at once.
- The evaluation run reported in Section 11 covers a single underlying model (gpt-4o-mini); the wider cost, speed and quality comparison across additional models has not yet been carried out.
- Agentic RAG's consistency score was pulled down by high run-to-run variation on 2 of the 10 test deals (Section 11.3); the underlying cause of that variation has not yet been diagnosed, and is a direct input to future tuning work.

13.3 Future Work

- Bring in real-time commodity or category price information, particularly for F&B raw materials, to reduce how much the system has to rely on user-supplied pricing alone.

- Add proper multi-user login in place of the current lightweight business identifier, as a prerequisite for serving more than one business account at a time.
- Track the actual, real-world outcome of each negotiation against the system's original prediction, closing the loop between what the tool recommends and how well it actually performs.
- Add WhatsApp or email export of the negotiation brief, alongside the existing PDF download, to match how small business owners in this segment tend to communicate.
- Add a voice-based rehearsal option so that practice can more closely resemble an actual spoken negotiation call.
- Build a shared supplier-profile record that tracks concession patterns for a named supplier across multiple businesses, with appropriate privacy safeguards, rather than only within one business's own history.
- Add Hindi, Tamil and Telugu language support for the Indian small-business market the knowledge base already targets.
- Extend the single-model evaluation in Section 11 to additional underlying models to complete the wider comparison originally planned, and diagnose the source of Agentic RAG's higher run-to-run variation on deals D02 and D08.